

Lecture 10: Enum, Encapsulation, and Any

Visibility modifiers and meaningful string forms

Agenda

1. `enum` class
2. Encapsulation and access modifiers
3. The `Any` class and `toString`

Why enums?

When a value is one of a **fixed set of named constants**, use an enum:

- Days, directions, order status, HTTP-ish app states
- Safer than raw strings/ints (typos become compile errors)
- In Kotlin: `enum class`

Basic enum

```
1 enum class Direction { NORTH, SOUTH, EAST, WEST }
2
3 fun move(d: Direction) = when (d) {
4     Direction.NORTH -> "up"
5     Direction.SOUTH -> "down"
6     Direction.EAST -> "right"
7     Direction.WEST -> "left"
8 }
```

when on enum can be exhaustive without else.

Enum with properties and methods

```
1 enum class Status(val code: Int) {  
2     READY(0), RUNNING(1), FAILED(2);  
3     fun isOk(): Boolean = this == READY || this == RUNNING  
4 }  
5  
6 fun main() {  
7     println(Status.FAILED.code)           // 2  
8     println(Status.READY.isOk())          // true  
9     println(Status.valueOf("RUNNING"))  
10 }
```

Enum extras

- `Status.values()` / `entries` (modern) — all constants
- `valueOf("NAME")` — parse; throws if unknown
- Implement interfaces on enums (advanced design)

Quiz 1 — Exhaustiveness

Quiz

For `enum class Color { R, G, B }`, which `when` needs no `else` to be a valid expression?

1. Branches only for R and G
2. Branches for R, G, and B
3. A single `else -> ...` only
4. `when` without subject

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 1

Answer

2. Cover all constants R, G, B.

Exhaustive enum `when` expressions compile without `else`.

Encapsulation

Hide representation; expose a controlled API.

- Protect invariants (balance ≥ 0 , non-blank id)
- Allow changing internals without breaking callers
- Kotlin visibility: `public` (default), `internal`, `protected`, `private`

Visibility at a glance

Modifier	Visible to
<code>public</code>	everywhere
<code>internal</code>	same module
<code>protected</code>	class + subclasses
<code>private</code>	class (or file if top-level)

Private state, public API

```
1 class Account(private var balance: Int) {  
2     fun deposit(amount: Int) {  
3         require(amount > 0)  
4         balance += amount  
5     }  
6     fun balance(): Int = balance  
7 }
```

Callers cannot assign `balance` directly — they go through methods.

Visibility on setters

```
1 class Person(name: String) {  
2     var name: String = name  
3     private set    // public get, private set  
4 }
```

Outside code can read `name` but cannot assign it.

Quiz 2 — Visibility

Quiz

A private member of class A is visible to:

1. Any class in the same package
2. Subclasses in other modules
3. Only code inside A (and nested types in A)
4. The whole module (`internal`-like)

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. Only inside A (including its nested classes).

The root type: `Any`

In Kotlin, every non-nullable class inherits (ultimately) from `Any`.

- Similar role to Java's `Object`
- Key open members: `equals`, `hashCode`, `toString`
- Nullable top is `Any?`

Default toString

```
1 class User(val id: Int, val name: String)
2
3 fun main() {
4     println(User(1, "Ada"))
5     // something like User@7a8c8d5 (not useful)
6 }
```

Override toString() for debugging and logs.

Custom toString

```
1 class User(val id: Int, val name: String) {  
2     override fun toString(): String =  
3         "User(id=$id, name=$name)"  
4 }
```

data class generates toString/equals/hashCode automatically — useful, but know how to write them manually too.

Polymorphic printing

```
1 open class Entity { override fun toString() = "Entity" }
2 class Order(val id: Int) : Entity() {
3     override fun toString() = "Order#$id"
4 }
5 fun show(x: Any) = println(x.toString())
```

Parameter type Any accepts anything; runtime toString runs.

Quiz 3 — toString dispatch

Quiz

With `val e: Entity = Order(9); println(e)`, what is printed (given previous slide)?

1. `Entity`
2. `Order#9`
3. Default `Order@...` identity string only
4. Compile error

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. `Order#9`

`println` calls `toString()`; override on `Order` is dispatched dynamically.

Summary

- `enum class` models closed constant sets; great with `when`
- Encapsulation via `private/protected/internal` and disciplined APIs
- Everything is an `Any`; override `toString` for readable output

Looking ahead

Next: abstract classes and interfaces — contracts without full implementations.